МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ КАЗАХСТАН

«НАО Евразийский национальный университет имени Л. Н. Гумилёва»

Кафедра Информационная безопасность

ОТЧЕТ

по практической работе № 5

Дисциплина: «Клиент-серверные технологии и их безопасность»

Тема: «CI/CD Security Pipeline: настройка автоматического сканирования уязвимостей исходного кода (SAST) и зависимостей (SCA) в GitHub Actions / GitLab CI»

Руководитель, преподаватель _____ Казиева Н.М.

«___» _____ 2026

Выполнил студент группы СИБ-24

Нуртазинов Ислам Тулегенович

«___» _____ 2026

Астана, 2026

1. Цель работы

Практическая: построить конвейер CI/CD, который при каждом изменении кода автоматически выполняет статический анализ исходного кода (SAST), анализ состава программного обеспечения и уязвимостей зависимостей (SCA), поиск секретов и тесты, а затем по формальной политике решает, можно ли принимать изменение.

Теоретическая: изучить подход DevSecOps и принцип «shift-left», классы уязвимостей, которые находят SAST и SCA, форматы обмена результатами (SARIF, отчёты GitLab) и ограничения автоматических сканеров.

2. Задание

Вариант — тема № 46 из перечня мини-проектов: «CI/CD Security Pipeline: Настройка автоматического сканирования уязвимостей исходного кода (SAST) и зависимостей (SCA) в GitHub Actions / GitLab CI».

Объектом проверки выбран REST-сервис с ролевой моделью доступа (RBAC) из методических указаний к практической работе № 5: таблицы users, roles, permissions, user_roles, role_permissions и эндпоинты /api/v1/public, /api/v1/moderator/audit, /api/v1/admin/users. Требуется:

- подготовить исходную версию сервиса с типичными ошибками безопасности в коде и устаревшими зависимостями;
- описать пайплайн безопасности для GitHub Actions и для GitLab CI: SAST (Bandit, Semgrep), SCA (pip-audit, Trivy), поиск секретов (Gitleaks / Secret Detection), тесты;
- написать собственные правила SAST для проверки RBAC и единый Security Gate с политикой блокировки;
- выполнить сканирование, проанализировать находки, устранить уязвимости и показать, что после исправления пайплайн проходит.

3. Теоретические сведения

3.1. DevSecOps и принцип «shift-left»

DevSecOps — практика встраивания проверок безопасности в каждый этап жизненного цикла разработки, а не в отдельный аудит перед выпуском. Принцип «shift-left» означает перенос проверок как можно ближе к моменту написания кода: уязвимость, найденная в pull request, исправляется за минуты, а найденная в эксплуатации — требует расследования инцидента, выпуска патча и уведомления пользователей. Пайплайн CI/CD — естественное место для таких проверок: он запускается при каждом изменении, одинаково для всех разработчиков, а его результат можно сделать обязательным условием слияния (branch protection).

3.2. SAST — статический анализ исходного кода

SAST (Static Application Security Testing) анализирует исходный код без его запуска: строит абстрактное синтаксическое дерево (AST), граф потока данных и ищет опасные конструкции — подстановку пользовательских данных в SQL-запрос или команду ОС, небезопасную десериализацию, слабые криптоалгоритмы, жёстко заданные секреты. Преимущества — полнота покрытия кода и точное указание файла и строки; недостатки — ложные срабатывания и слепота к ошибкам бизнес-логики, не описанным правилами (например, к забытой проверке роли). Последнее компенсируется собственными правилами.

Bandit — анализатор Python от PyCQA: набор плагинов B1xx–B7xx по AST, у каждой находки есть уровень серьёзности (severity) и достоверности (confidence).

Semgrep — многоязычный анализатор с правилами в виде шаблонов кода (YAML). Поддерживает наборы правил сообщества (p/python, p/owasp-top-ten) и позволяет быстро писать собственные правила под архитектуру проекта.

3.3. SCA — анализ состава ПО и зависимостей

SCA (Software Composition Analysis) строит перечень сторонних компонентов приложения, включая транзитивные зависимости, и сопоставляет их версии с базами известных уязвимостей: CVE/NVD, GitHub Advisory Database (GHSA), Python Packaging Advisory Database (PYSEC) и агрегатором OSV. Для каждой уязвимости известна оценка CVSS и версия, в которой она исправлена. Это соответствует категории OWASP Top 10 A06:2021 «Vulnerable and Outdated Components». Особенность SCA — результат меняется без изменения кода: новая CVE в используемой библиотеке появляется в любой день, поэтому пайплайн запускается и по расписанию.

pip-audit (PyPA) разрешает полное дерево зависимостей requirements.txt и проверяет его по базе PyPI/OSV. **Trivy** (Aqua Security) — универсальный сканер файловой системы, образов контейнеров и IaC с собственной базой уязвимостей.

3.4. Поиск секретов и формат SARIF

Утечка ключей и паролей в репозиторий — отдельный класс проблем: секрет остаётся в истории git даже после удаления из текущей версии. Gitleaks и GitLab Secret Detection проверяют всю историю коммитов по сигнатурам и энтропии строк.

SARIF (Static Analysis Results Interchange Format, OASIS) — стандартный JSON-формат результатов статического анализа. GitHub принимает SARIF через действие upload-sarif и показывает находки во вкладке Security → Code scanning и прямо в diff pull request. GitLab использует собственные форматы отчётов (artifacts:reports:sast, dependency_scanning), которые отображаются в виджете Merge Request и в Security Dashboard.

Таблица 1 — Сравнение видов автоматического тестирования безопасности

Подход	Что анализирует	Когда	Примеры находок	Инструменты
SAST	исходный код	commit / PR	SQLi, OS command	Bandit, Semgrep,

Подход	Что анализирует	Когда	Примеры находок	Инструменты
	(AST, потоки данных)		injection, слабый хеш, секрет в коде	CodeQL
SCA	зависимости и их версии	commit / PR + расписание	CVE в библиотеках, лицензии	pip-audit, Trivy, Dependabot
Secrets	история git	commit / PR	API-ключи, пароли, токены	Gitleaks, Secret Detection
DAST	работающее приложение	после деплоя на стенд	ошибки конфигурации, XSS, IDOR	OWASP ZAP, Burp

4. Среда выполнения и инструменты

Таблица 2 — Среда выполнения и версии инструментов

Компонент	Версия / значение
ОС (локальный прогон)	Windows 11 Pro
Python (локально / в CI)	3.10.7 / 3.12
FastAPI · PyJWT · PyYAML (после исправления)	0.141.1 · 2.15.1 · 6.0.3
pytest	9.2.0
Bandit	1.9.4
Semgrep	1.183.0
pip-audit	2.10.1
Trivy (в CI)	aquasecurity/trivy-action v0.36.0, образ aquasec/trivy:0.67.2
Gitleaks (в CI / локально)	gitleaks/gitleaks-action@v3 / gitleaks 8.30.1
Репозиторий GitHub	github.com/is1am/cicd-security-pipeline
Система контроля версий	git, ветка main

Таблица 3 — Структура проекта ПР5_CICD_Security

Путь	Назначение
app/main.py, app/security.py, app/db.py	RBAC-сервис на FastAPI (объект проверки)
tests/test_rbac.py	функциональные и негативные тесты RBAC
.github/workflows/security.yml	пайплайн GitHub Actions
.github/dependabot.yml	автоматические PR с обновлением зависимостей
.gitlab-ci.yml	пайплайн GitLab CI
.semgrep/rbac-rules.yml	собственные правила Semgrep
security-policy.json	политика Security Gate (пороги и исключения)
.gitleaks.toml	стандартные правила Gitleaks + исключение для reports/
scripts/security_gate.py	сводка отчётов и решение «пропустить / заблокировать»
scripts/run_local_scans.py	локальный прогон тех же проверок, что и в CI
reports/before, reports/after	отчёты сканеров до и после исправлений

5. Объект проверки: RBAC-сервис

Сервис реализует классическую модель RBAC из методических указаний: права (permissions) назначаются ролям, роли — пользователям, а каждый эндпоинт защищён зависимостью FastAPI require_permission(...),

которая возвращает 403 Forbidden при отсутствии права. Схема БД создаётся с внешними ключами и ON DELETE CASCADE.

Таблица 4 — Эндпоинты сервиса и права доступа

Эндпоинт	Требуемое право	Роли
POST /api/v1/auth/login	—	все (выдача JWT)
GET /api/v1/public	READ_PUBLIC	USER, MODERATOR, ADMIN
GET /api/v1/users/search	READ_PUBLIC	USER, MODERATOR, ADMIN
GET /api/v1/moderator/audit	VIEW_AUDIT	MODERATOR, ADMIN
GET /api/v1/moderator/audit/export	EXPORT_AUDIT	ADMIN
POST /api/v1/admin/users	MANAGE_USERS	ADMIN
POST /api/v1/admin/import	MANAGE_USERS	ADMIN

В исходную версию (коммит a4be20f) намеренно внесены ошибки, которые часто встречаются в реальных проектах:

Таблица 5 — Уязвимости исходной версии

№	Ошибка	Где	CWE / OWASP
1	SQL-запрос собирается f-строкой из параметра q	main.py, /users/search	CWE-89 · A03 Injection
2	Вызов оболочки ОС с shell=True и параметром запроса	main.py, /audit/export	CWE-78 · A03 Injection
3	Эндпоинт экспорта не проверяет право EXPORT_AUDIT	main.py, /audit/export	CWE-285 · A01 Broken Access Control
4	yaml.load с yaml.Loader (создание произвольных объектов)	main.py, /admin/import	CWE-502 · A08
5	Секрет подписи JWT в исходном коде	security.py	CWE-798 · A07
6	JWT декодируется без проверки подписи	security.py	CWE-347 · A02
7	Пароли хешируются MD5 без соли	security.py	CWE-327/916 · A02
8	Сервер слушает 0.0.0.0	main.py	CWE-605 · A05
9	Устаревшие версии fastapi, PyJWT, requests, Jinja2	requirements.txt	A06 Vulnerable Components

6. Реализация пайплайна безопасности

6.1. Архитектура

Пайплайн запускается при push и pull request в ветку main, еженедельно по расписанию (чтобы обнаруживать новые CVE в неизменном коде) и вручную. Проверки выполняются параллельно в отдельных джобах, каждая сохраняет машиночитаемый отчёт (JSON) и, где возможно, SARIF. Финальная джоба Security Gate собирает отчёты и по политике принимает решение. В настройках репозитория (branch protection) статус Security Gate

делается обязательным, поэтому изменение с блокирующей находкой нельзя слить.

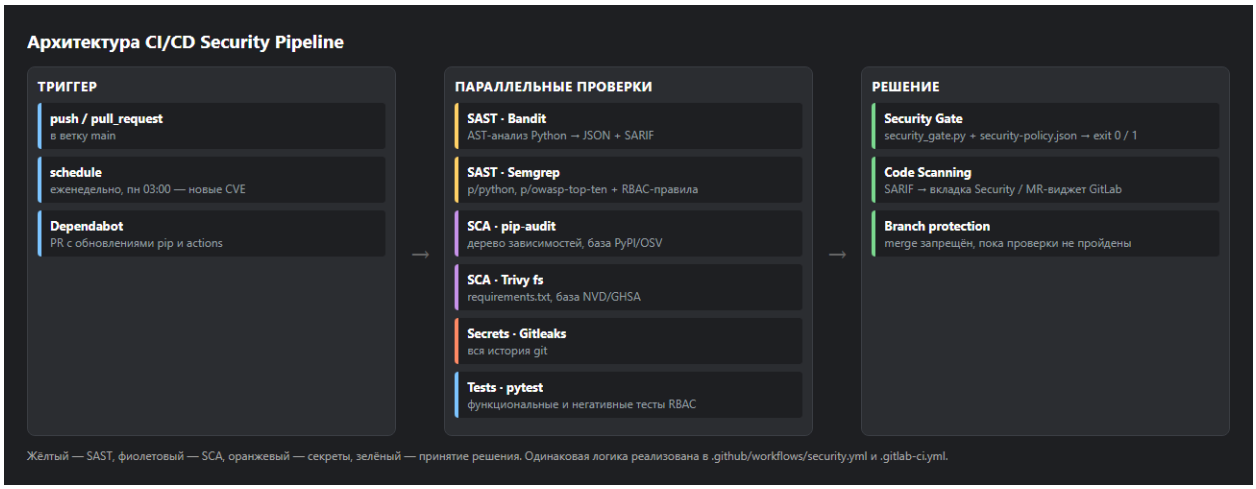


Рисунок 1 — Архитектура CI/CD Security Pipeline

6.2. GitHub Actions

Файл .github/workflows/security.yml. Основные решения:

- `permissions: contents: read` на уровне workflow и `security-events: write` только в джобах, загружающих SARIF — принцип наименьших привилегий для GITHUB_TOKEN;
- версии сканеров и Docker-образов зафиксированы (воспроизводимость, защита от подмены инструмента); для production рекомендуется закреплять actions по SHA коммита;
- Bandit, Semgrep и Trivy загружают SARIF в Code Scanning (category разделяет инструменты);
- Gitleaks получает полную историю (fetch-depth: 0);
- сканеры не прерывают джобу сами (--exit-zero / || true): решение принимает Security Gate по единой политике, а отчёты сохраняются как артефакты даже при наличии находок;
- `concurrency` отменяет устаревшие запуски той же ветки.

```
  sast-semgrep:
    name: SAST · Semgrep
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write
    container:
      image: semgrep/semgrep:1.183.0
    steps:
      - uses: actions/checkout@v7
      - name: Semgrep (стандартные наборы + собственные RBAC-правила)
        run: >
          semgrep scan --metrics=off
          --config p/python --config p/owasp-top-ten
          --config .semgrep/rbac-rules.yml
          --json -o semgrep.json
          --sarif-output=semgrep.sarif
          app
      - uses: github/codeql-action/upload-sarif@v4
        if: always()
        with:
          sarif_file: semgrep.sarif
          category: semgrep
```

Листинг 1 — джоба Semgrep в GitHub Actions

```
  security-gate:
    name: Security Gate
    runs-on: ubuntu-latest
    needs: [sast-bandit, sast-semgrep, sca-pip-audit, sca-trivy]
    if: always()
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: ${{ env.PYTHON_VERSION }}
      - uses: actions/download-artifact@v8
        with:
          path: reports
          merge-multiple: true
      - name: Сводка и решение о пропуске сборки
        run: python scripts/security_gate.py reports --policy security-policy.json --
require-trivy --summary "$GITHUB_STEP_SUMMARY"
```

Листинг 2 — джоба Security Gate в GitHub Actions

6.3. GitLab CI

Файл .gitlab-ci.yml реализует ту же логику в стадиях test → security → gate. Встроенный шаблон Jobs/Secret-Detection подключается через include. Semgrep формирует отчёт в формате GitLab SAST (--gitlab-sast-output), Trivy — отчёт dependency_scanning по шаблону gitlab.tpl; оба публикуются через artifacts:reports и отображаются в Merge Request. Собственные джобы не требуют тарифа Ultimate.

```
sast:semgrep:
  stage: security
  image: $SEMGREP_IMAGE
  script:
    - >
      semgrep scan --metrics=off
      --config p/python --config p/owasp-top-ten
      --config .semgrep/rbac-rules.yml
      --json -o semgrep.json
      --gitlab-sast-output=gl-sast-report.json
      app
  artifacts:
    when: always
    paths: [semgrep.json]
    reports:
      sast: gl-sast-report.json
```

Листинг 3 — джоба Semgrep в GitLab CI

```
sca:trivy:
  stage: security
  image:
    name: $TRIVY_IMAGE
    entrypoint: [""]
  script:
    - trivy fs --scanners vuln --format json -o trivy.json .
    - trivy fs --scanners vuln --format template
        --template "@/contrib/gitlab.tpl" -o gl-dependency-scanning-report.json .
  artifacts:
    when: always
    paths: [trivy.json]
    reports:
      dependency_scanning: gl-dependency-scanning-report.json
```

Листинг 4 — джоба Trivy в GitLab CI

6.4. Собственные правила Semgrep для RBAC

Стандартные правила не знают, что в проекте права проверяются через require_permission. Поэтому написано правило rbac-privileged-route-without-permission: оно срабатывает на любой маршрут /api/v1/admin/* или /api/v1/moderator/*, у которого нет зависимости Depends(require_permission(...)) ни в параметрах функции, ни в dependencies декоратора. Так SAST начинает находить ошибку контроля доступа (OWASP API5 Broken Function Level Authorization), которую обычные сканеры пропускают. Ещё два правила запрещают декодирование JWT без проверки подписи и секрет в коде.

```
  - id: rbac-privileged-route-without-permission
    message: >-
      Привилегированный эндпоинт $PATH не проверяет разрешение через
      require_permission(...). Достаточно любого валидного токена —
      возможна эскалация привилегий (OWASP API5: Broken Function Level Authorization).
    severity: ERROR
    languages: [python]
    metadata:
      cwe: "CWE-285: Improper Authorization"
      owasp: "A01:2021 - Broken Access Control"
      category: security
    patterns:
      - pattern: |
          @$APP.$METHOD($PATH, ...)
          def $FUNC(...):
            ...
      - metavariable-regex:
          metavariable: $METHOD
          regex: ^(get|post|put|patch|delete)$
      - metavariable-regex:
          metavariable: $PATH
          regex: ^["']/api/v1/(admin|moderator)
      - pattern-not: |
          @$APP.$METHOD($PATH, ...)
          def $FUNC(..., $U = Depends(require_permission(...)), ...):
            ...
      - pattern-not: |
          @$APP.$METHOD($PATH, ..., dependencies=[..., Depends(require_permission(...)),
...], ...)
          def $FUNC(...):
            ...
```

Листинг 5 — правило Semgrep для проверки RBAC-маршрутов

6.5. Security Gate и политика

Скрипт scripts/security_gate.py читает bandit.json, semgrep.json, pip-audit.json и trivy.json, нормализует находки, устраняет дубли (pip-audit выдаёт одну уязвимость под несколькими идентификаторами — GHSA, PYSEC, CVE) и применяет политику security-policy.json. Сборка блокируется, если есть хотя бы одна блокирующая находка или отсутствует отчёт одного из сканеров (сломанный сканер не должен означать «уязвимостей нет»). Сводка в Markdown публикуется в GITHUB_STEP_SUMMARY.

```
{
  "_comment": "Политика Security Gate: что блокирует слияние. Исключения — только с
обоснованием и сроком.",
  "block": {
    "bandit_min_severity": "MEDIUM",
    "bandit_min_confidence": "MEDIUM",
    "semgrep_severities": ["ERROR", "WARNING"],
    "sca_block_if_fix_available": true,
    "trivy_severities": ["CRITICAL", "HIGH", "MEDIUM"]
  },
  "ignore": [
    {
      "id": "B105",
      "path": "app/main.py",
      "reason": "Ложное срабатывание: строка 'bearer' — тип токена OAuth 2.0, не пароль",
      "expires": "2027-06-30"
    }
  ]
}
```

Листинг 6 — политика security-policy.json

Исключение допускается только адресно (идентификатор правила + файл), с обоснованием и сроком действия: по истечении срока находка снова блокирует сборку. Это защищает от бесконтрольного накопления «временных» подавлений.

7. Результаты сканирования исходной версии

Все проверки выполнены командой python scripts/run_local_scans.py before — она запускает те же инструменты с теми же параметрами, что и джобы пайплайна, и сохраняет отчёты в reports/before.

7.1. Тесты

Из пяти тестов три упали: пользователь с ролью USER получил доступ к экспорту аудита (200 вместо 403), токен, подписанный ключом злоумышленника, принят (200 вместо 401), а запрос с параметром zzz' OR 1=1 -- вернул всех пользователей.

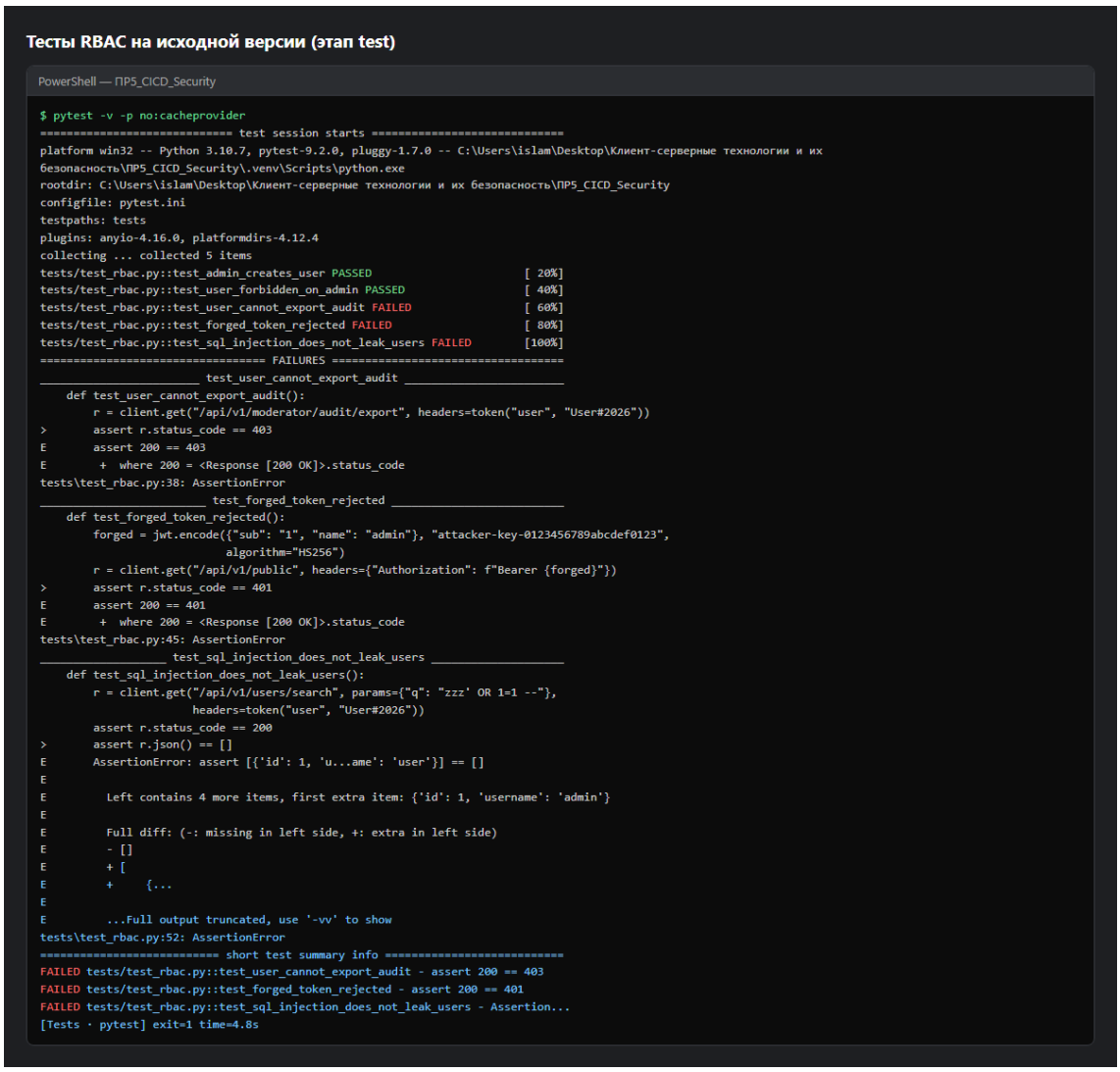


Рисунок 2 — pytest на исходной версии: 3 из 5 тестов не пройдены

7.2. SAST: Bandit

Bandit: исходная версия

PowerShell — ПР5_CICD_Security

```
$ bandit -r app -q
Run started:2026-11-02 14:28:28.557864+00:00
Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   Location: app\main.py:6:0
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'bearer'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   Location: app\main.py:57:70
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   Location: app\main.py:70:12
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   Location: app\main.py:82:10
>> Issue: [B506:yaml_load] Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider yaml.safe_load().
   Severity: Medium   Confidence: High
   CWE: CWE-20 (https://cwe.mitre.org/data/definitions/20.html)
   Location: app\main.py:95:11
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
   Severity: Medium   Confidence: Medium
   CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
   Location: app\main.py:102:26
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'rbac-super-secret-key-2026'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   Location: app\security.py:10:13
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   Location: app\security.py:15:11
```

Рисунок 3 — Bandit: находки в исходной версии

Bandit нашёл 8 проблем, из них 5 блокирующих по политике (уровень и достоверность не ниже MEDIUM): SQL-инъекция (B608), shell=True (B602), yaml.load (B506), привязка к 0.0.0.0 (B104), MD5 (B324). Находки уровня LOW (импорт subprocess, строка «bearer») не блокируют сборку; «bearer» — ложное срабатывание, внесённое в исключения политики. Bandit отметил секрет в коде только как LOW и не обнаружил ни отключённую проверку подписи JWT, ни отсутствие проверки права на эндпоинте экспорта.

7.3. SAST: Semgrep

Semgrep: результаты сканирования (semgrep.json, before)

Файл:строка	Правило	Уровень	CWE
app/main.py:79	rbac-privileged-route-without-permission	ERROR	CWE-285: Improper Authorization
app/main.py:82	subprocess-shell-true	ERROR	CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
app/main.py:95	avoid-pyyaml-load	ERROR	CWE-502: Deserialization of Untrusted Data
app/security.py:10	hardcoded-jwt-secret	ERROR	CWE-798: Use of Hard-coded Credentials
app/security.py:15	insecure-hash-algorithm-md5	WARNING	CWE-327: Use of a Broken or Risky Cryptographic Algorithm
app/security.py:15	md5-used-as-password	WARNING	CWE-327: Use of a Broken or Risky Cryptographic Algorithm
app/security.py:26	jwt-python-hardcoded-secret	ERROR	CWE-522: Insufficiently Protected Credentials
app/security.py:31	jwt-decode-without-verification	ERROR	CWE-347: Improper Verification of Cryptographic Signature
app/security.py:31	unverified-jwt-decode	ERROR	CWE-287: Improper Authentication

Найдено: **9** · проверено файлов: 4 · наборы: p/python, p/owasp-top-ten, .semgrep/rbac-rules.yml

Рисунок 4 — Semgrep: находки в исходной версии

Semgrep нашёл 9 проблем — все блокирующие. Собственные правила обнаружили маршрут /api/v1/moderator/audit/export без require_permission (строка 79), декодирование JWT без проверки подписи и секрет в коде. Правила сообщества дополнительно указали на shell=True, yaml.load и MD5 для паролей. Bandit и Semgrep частично пересекаются, но каждый нашёл то, что пропустил другой, — поэтому в пайплайне оба.

7.4. SCA: pip-audit

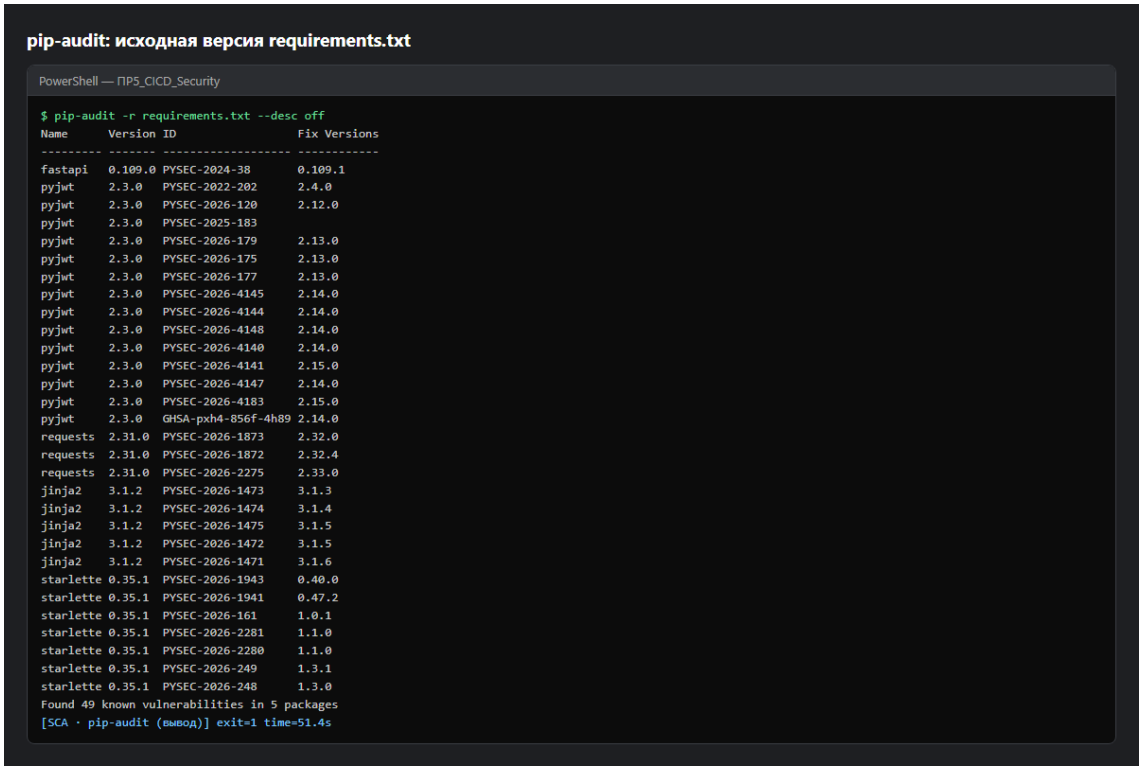


Рисунок 5 — pip-audit: уязвимые зависимости исходной версии

Таблица 6 — Уязвимости зависимостей исходной версии (после удаления дублей)

Пакет	Зависимость	Уязв.	Примеры идентификаторов	Исправлено в
fastapi 0.109.0	прямая	1	CVE-2024-24762	0.109.1
pyjwt 2.3.0	прямая	14	CVE-2022-29217, CVE-2026-32597, CVE-2025-45768 …	2.15.0
requests 2.31.0	прямая	3	CVE-2024-35195, CVE-2024-47081, CVE-2026-25645	2.33.0
jinja2 3.1.2	прямая	5	CVE-2024-22195, CVE-2024-34064, CVE-2024-56326 …	3.1.6
starlette 0.35.1	транзитивная (через fastapi)	7	CVE-2024-47874, CVE-2025-54121, CVE-2026-48710 …	1.3.1

pip-audit вывел 49 записей, после удаления дублей — 30 уникальных уязвимостей в 5 пакетах; 29 из них имеют исправленную версию и

блокируют сборку. Важно, что starlette в requirements.txt не указан: это транзитивная зависимость fastapi, и её уязвимости (в т.ч. CVE-2024-47874 — отказ в обслуживании при разборе multipart) видны только при анализе полного дерева.

7.5. Решение Security Gate

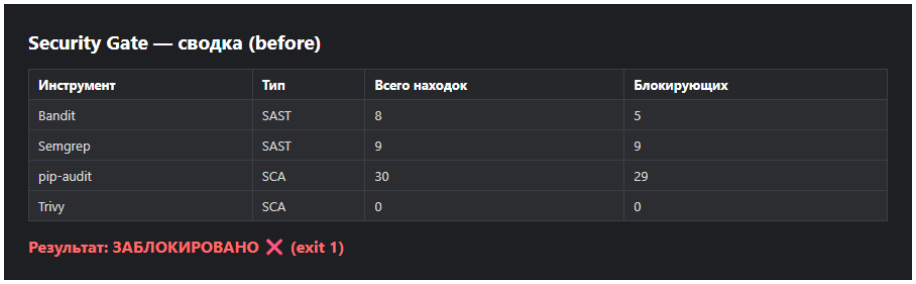


Рисунок 6 — Security Gate для исходной версии: сборка заблокирована

Итого блокирующих находок: 43. Gate завершился с кодом 1, что в CI означает красный статус и запрет слияния.

8. Устранение уязвимостей

Исправления внесены отдельным коммитом 836b494. Соответствие находок и исправлений приведено в таблице.

Таблица 7 — Исправления по результатам сканирования

Находка	Инструмент	Исправление
SQL-инъекция (B608)	Bandit, тест	параметризованный запрос с плейсхолдером ?
shell=True (B602)	Bandit, Semgrep	CSV формируется модулем csv, оболочка не вызывается
Маршрут без проверки права	Semgrep (RBAC-правило), тест	Depends(require_permission("EXPORT_AUDIT"))
yaml.load (B506)	Bandit, Semgrep	yaml.safe_load
Секрет в коде	Semgrep	JWT_SECRET из окружения, проверка длины ≥ 32
JWT без проверки подписи	Semgrep, тест	jwt.decode(key, algorithms=["HS256"], require=[sub, exp])
MD5 для паролей (B324)	Bandit, Semgrep	PBKDF2-HMAC-SHA256, соль 16 байт, 600 000 итераций, compare_digest
Привязка к 0.0.0.0 (B104)	Bandit	HOST из окружения, по умолчанию 127.0.0.1
Уязвимые зависимости	pip-audit	обновление до актуальных версий, Dependabot

```
# Было
def decode_token(token: str) -> dict:
    # подпись не проверяется — можно подделать любой токен
    return jwt.decode(token, options={"verify_signature": False})


# Стало
def decode_token(token: str) -> dict:
    # подпись и срок действия проверяются, алгоритм — из белого списка
    return jwt.decode(token, SIGNING_KEY, algorithms=[ALGORITHM],
                      options={"require": ["sub", "exp"]})
```

Листинг 7 — проверка JWT до и после исправления

```
# Было
        # SQL-инъекция: параметр подставляется в строку запроса
        rows = conn.execute(
            f"SELECT id, username FROM users WHERE username LIKE '%{q}%'").fetchall()


# Стало
        # параметризованный запрос: значение передаётся отдельно от SQL
        rows = conn.execute("SELECT id, username FROM users WHERE username LIKE ?",
                            (f"%{q}%",)).fetchall()
```

Листинг 8 — поиск пользователей до и после исправления

```
# Было
@app.get("/api/v1/moderator/audit/export")
def audit_export(fmt: str = "csv", user=Depends(get_current_user)):
    # нет проверки разрешения EXPORT_AUDIT + инъекция команд ОС
    out = subprocess.check_output(f"echo audit export as {fmt}", shell=True)
    return {"result": out.decode().strip()}


# Стало
@app.get("/api/v1/moderator/audit/export")
def audit_export(user=Depends(require_permission("EXPORT_AUDIT"))):
    # экспорт формируется средствами Python, без вызова командной оболочки
    buf = io.StringIO()
    csv.writer(buf).writerows([["event"], ["login admin"], ["login moder"]])
    return {"result": buf.getvalue()}
```

Листинг 9 — эндпоинт экспорта аудита до и после исправления

Таблица 8 — Изменение версий в requirements.txt

Пакет	**Было**	**Стало**
fastapi	0.109.0	0.141.1
uvicorn	0.27.0	0.54.0
pydantic	2.5.3	2.14.2
PyJWT	2.3.0	2.15.1
PyYAML	6.0.1	6.0.3
requests	2.31.0	2.34.2
Jinja2	3.1.2	3.1.6

9. Повторное сканирование

Команда python scripts/run_local_scans.py after выполнена на исправленной версии, отчёты сохранены в reports/after.

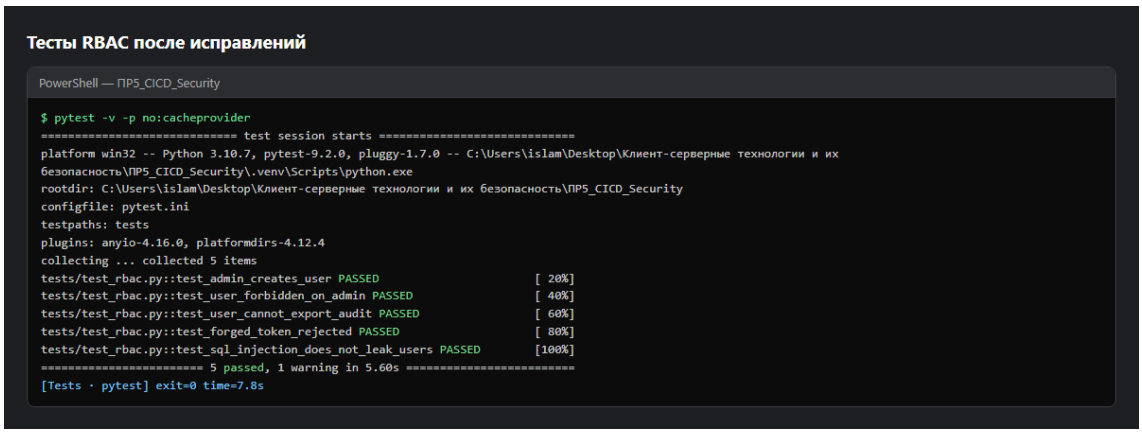


Рисунок 7 — pytest после исправлений: 5 из 5 тестов пройдены

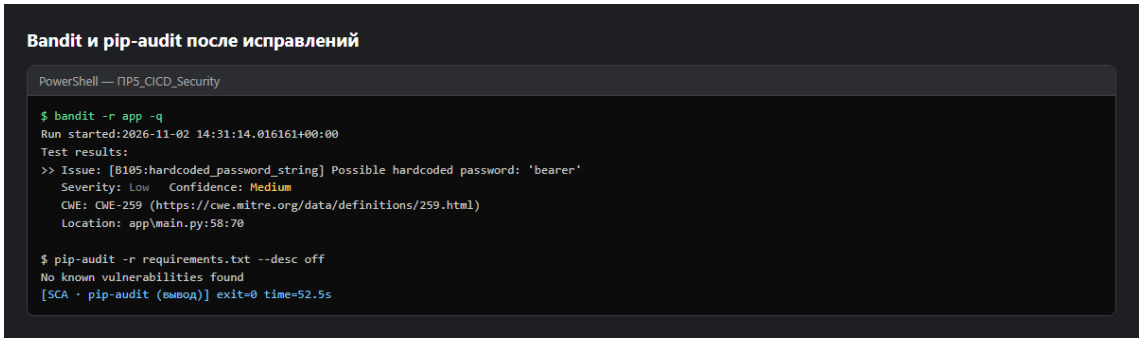


Рисунок 8 — Bandit и pip-audit после исправлений

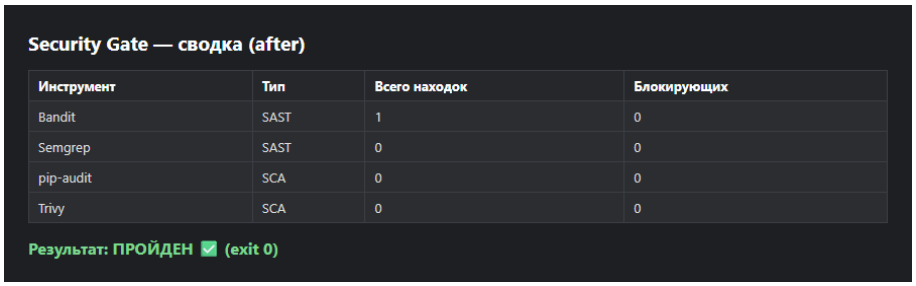


Рисунок 9 — Security Gate после исправлений: проверка пройдена

Bandit сообщает только о строке «bearer» уровня LOW — это тип токена OAuth 2.0, ложное срабатывание, внесённое в исключения политики с обоснованием и сроком. Semgrep и pip-audit находок не выдали.

Таблица 9 — Сравнение результатов до и после исправлений

Показатель	**До**	**После**
Тесты pytest (пройдено из 5)	2	5
Bandit: всего / блокирующих	8 / 5	1 / 0
Semgrep: всего / блокирующих	9 / 9	0 / 0
pip-audit: уникальных CVE / блокирующих	30 / 29	0 / 0
Решение Security Gate	ЗАБЛОКИРОВАНО (exit 1)	ПРОЙДЕН (exit 0)

9.1. Запуск в GitHub Actions

Проект опубликован в публичном репозитории github.com/is1am/cicd-security-pipeline. Первый запуск пайплайна (run #1) завершился с ошибкой, и разбор его результатов оказался полезным сам по себе.

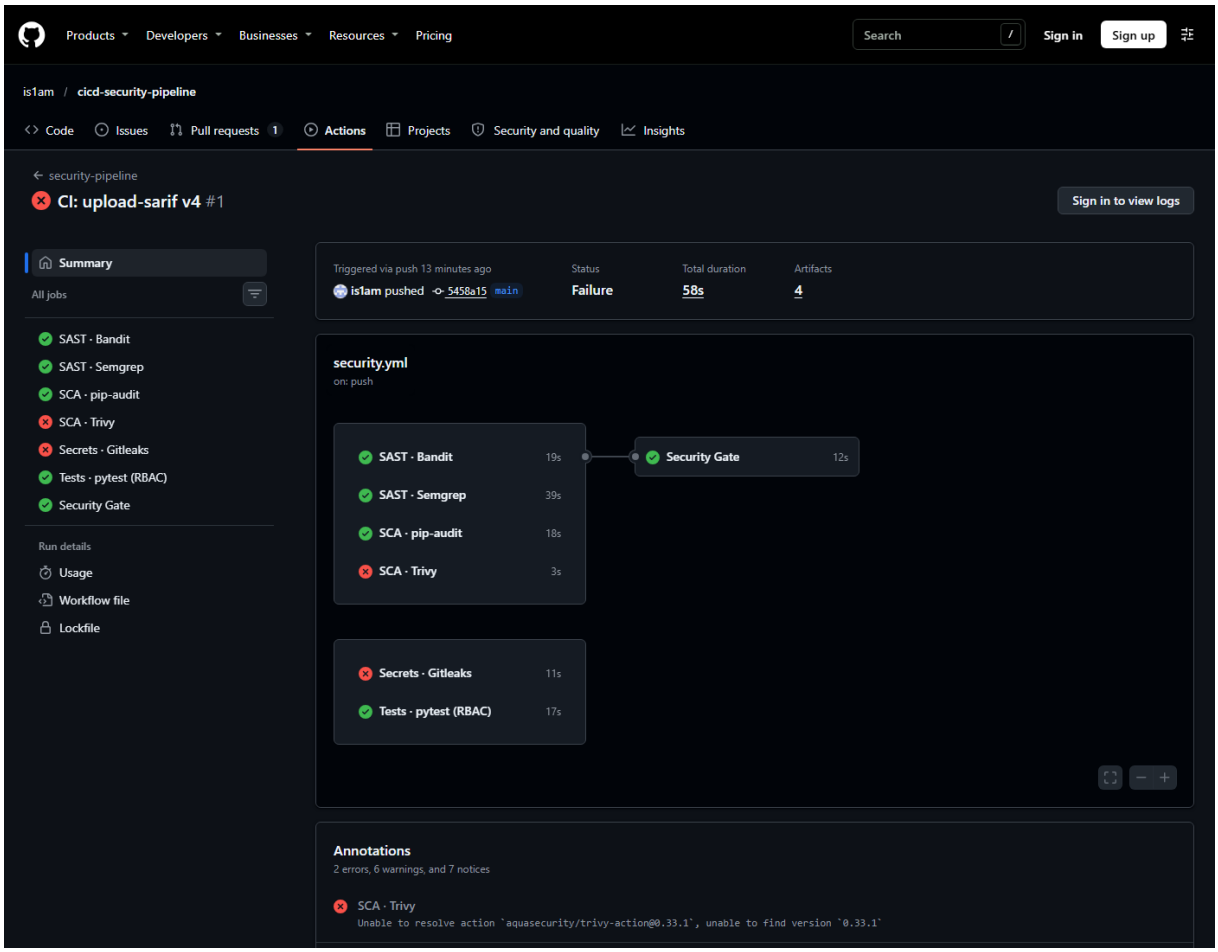


Рисунок 10 — Первый запуск в GitHub Actions (run #1): Trivy и Gitleaks не пройдены

Таблица 10 — Разбор первого запуска в GitHub Actions

Джоба	Результат	Причина и исправление
SCA · Trivy	ошибка	версия action указана как 0.33.1 вместо тега v0.33.1 — GitHub не нашёл action; исправлено на aquasecurity/trivy-action@v0.36.0
Secrets · Gitleaks	6 находок	приватный EC-ключ, JWT и generic-api-key в reports/before/pip-audit.json — это примеры из текстов advisory PyJWT, попавшие в закоммиченный отчёт сканера (ложное срабатывание); добавлен .gitleaks.toml с исключением для каталога reports/
Security Gate	пройден (ошибка Gate)	отчёт trivy.json отсутствовал, но Gate считал его необязательным; добавлен флаг --require-trivy, в CI отсутствие отчёта Trivy теперь блокирует сборку
Все джобы	предупреждения	actions на Node 20 устарели; обновлены до checkout@v7, setup-python@v7, upload-artifact@v7, download-artifact@v8, gitleaks-action@v3

Отдельно стоит отметить результат Gitleaks: строку rbac-super-secret-key-2026, которая действительно была секретом в исходной версии, он не обнаружил — её энтропия слишком низка для его эвристик. Этот секрет нашли только Semgrep (собственное правило hardcoded-jwt-secret и правило сообщества) и, с низким уровнем, Bandit. Это ещё один довод в пользу сочетания разных классов сканеров. Найденный в истории git секрет в реальном проекте нужно считать скомпрометированным и заменить, а не только удалить из текущей версии кода.

Второй запуск (run #8, после исправлений) прошёл полностью: все семь джоб завершились успешно, Security Gate разрешил сборку. Dependabot, включённый файлом .github/dependabot.yml, сразу после публикации открыл pull request «Bump fastapi from 0.141.1 to 0.162.0» — SCA работает и вне пайплайна.

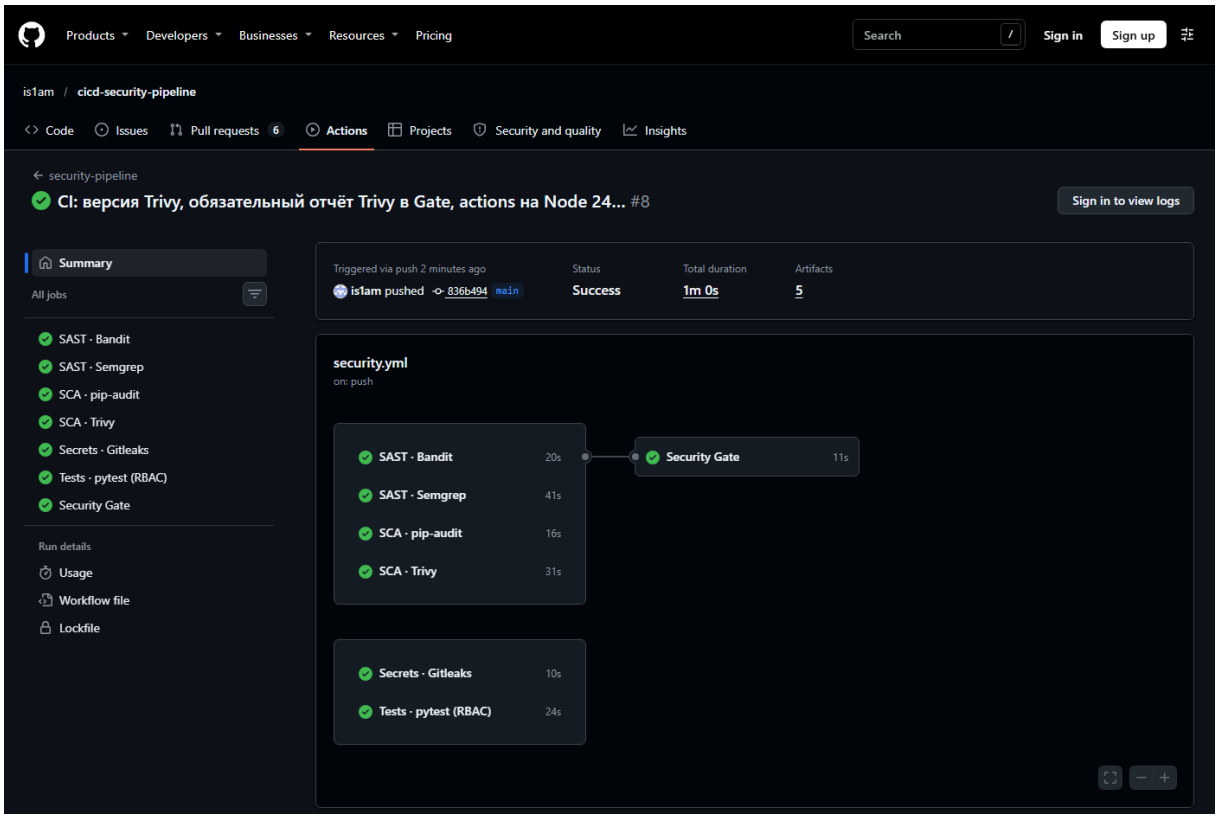


Рисунок 11 — Запуск в GitHub Actions после исправлений (run #8): все проверки пройдены

10. Сравнение GitHub Actions и GitLab CI

Таблица 11 — Реализация пайплайна в двух CI-системах

Критерий	GitHub Actions	GitLab CI

Критерий	GitHub Actions	GitLab CI
Описание пайплайна	.github/workflows/*.yml, jobs + needs	.gitlab-ci.yml, stages + needs
Готовые компоненты	Marketplace: trivy-action, gitleaks-action, upload-sarif	шаблоны include: SAST, Secret-Detection, Dependency-Scanning
Показ находок	SARIF → Security / Code scanning, аннотации в PR	artifacts:reports → виджет MR, Security Dashboard
Ограничения тарифа	Code scanning бесплатен для публичных репозиториев	Dependency Scanning и Security Dashboard — Ultimate; собственные джобы — на любом
Права токена	permissions на уровне workflow и job	CI_JOB_TOKEN, protected-переменные
Обновление зависимостей	Dependabot	Renovate / Dependency Scanning
Блокировка слияния	branch protection: required status checks	«Pipelines must succeed», approval policies

11. Выводы

1. Построен конвейер безопасности для GitHub Actions и GitLab CI, который при каждом изменении и еженедельно по расписанию выполняет SAST (Bandit, Semgrep), SCA (pip-audit, Trivy), поиск секретов (Gitleaks / Secret Detection) и тесты. Решение о допуске изменения принимает единый Security Gate по явной политике.

2. На исходной версии RBAC-сервиса пайплайн обнаружил 43 блокирующих находок: инъекции SQL и команд ОС, небезопасную десериализацию, секрет в коде, отключённую проверку подписи JWT, MD5 для паролей и 30 известных уязвимостей в 5 пакетах, включая транзитивную зависимость starlette. После исправлений все проверки пройдены.

3. Ни один инструмент не нашёл всё в одиночку: Bandit не заметил отключённую проверку подписи JWT и отсутствие проверки права; правила сообщества Semgrep не знают о require_permission. Комбинация нескольких сканеров и собственных правил, учитывающих архитектуру проекта, существенно повышает полноту анализа.

4. Для RBAC-систем особенно важно собственное правило SAST, проверяющее, что каждый привилегированный маршрут защищён проверкой права. Оно превращает ошибку контроля доступа (эскалацию привилегий) из находки пентеста в автоматическую проверку каждого pull request.

Негативные тесты (403 для USER, 401 для поддельного токена) дополняют SAST, подтверждая поведение на работающем приложении.

5. SCA должен анализировать полное дерево зависимостей и запускаться по расписанию: уязвимости появляются в библиотеках без изменения кода проекта. Автоматические PR Dependabot сокращают время от публикации CVE до обновления.

6. Эффективность пайплайна определяют не только сканеры, но и процесс: обязательный статус в branch protection, минимальные права токена CI, зафиксированные версии инструментов, блокировка при отсутствии отчёта и исключения только с обоснованием и сроком действия.